

条款 38：通过复合塑模出 **has-a** 或“根据某物实现出”

Model "has-a" or "is-implemented-in-terms-of" through composition.

复合（composition）是类型之间的一种关系，当某种类型的对象内含它种类型的对象，便是这种关系。例如：

```
class Address { ... };           //某人的住址
class PhoneNumber { ... };

class Person {
public:
    ...
private:
    std::string name;           //合成成分物（composed object）
    Address address;           //同上
    PhoneNumber voiceNumber;    //同上
    PhoneNumber faxNumber;      //同上
};
```

本例之中 Person 对象由 string, Address, PhoneNumber 构成。在程序员之间复合（composition）这个术语有许多同义词，包括 *layering*（分层），*containment*（内含），*aggregation*（聚合）和 *embedding*（内嵌）。

条款 32 曾说，“public 继承”带有 **is-a**（是一种）的意义。复合也有它自己的意义。实际上它有两个意义。复合意味 **has-a**（有一个）或 **is-implemented-in-terms-of**（根据某物实现出）。那是因为你正打算在你的软件中处理两个不同的领域（domains）。程序中的对象其实相当于你所塑造的世界中的某些事物，例如人、汽车、一张张视频画面等等。这样的对象属于应用域（*application domain*）部分。其他对象则纯粹是实现细节上的人工制品，像是缓冲区（*buffers*）、互斥器（*mutexes*）、查找树（*search trees*）等等。这些对象相当于你的软件的实现域（*implementation domain*）。当复合发生于应用域内的对象之间，表现出 **has-a** 的关系；当它发生于实现域内则是表现 **is-implemented-in-terms-of** 的关系。

上述的 Person class 示范 **has-a** 关系。Person 有一个名称，一个地址，以及语音和传真两笔电话号码。你不会说“人是一个名称”或“人是一个地址”，你会说“人有一个名称”和“人有一个地址”。大多数人接受此一区别毫无困难，所以很少人会对 **is-a** 和 **has-a** 感到困惑。

比较麻烦的是区分 **is-a**（是一种）和 **is-implemented-in-terms-of**（根据某物实现出）这两种对象关系。假设你需要一个 **template**，希望制造出一组 **classes** 用来表现由不重复对象组成的 **sets**。由于复用（*reuse*）是件美妙无比的事情，你的第一个

直觉是采用标准程序库提供的 `set` `template`。是的, 如果他人所写的 `template` 合乎需求, 我们何必另写一个呢?

不幸的是 `set` 的实现往往招致“每个元素耗用三个指针”的额外开销。因为 `sets` 通常以平衡查找树 (`balanced search trees`) 实现而成, 使它们在查找、安插、移除元素时保证拥有对数时间 (`logarithmic-time`) 效率。当速度比空间重要, 这是个通情达理的设计, 但如果你的程序却是空间比速度重要呢? 那么标准程序库的 `set` 提供给你的是个错误决定下的取舍。似乎你终究还得写个自己的 `template`。

但是容我再说一次, 复用 (`reuse`) 是件美好的事。如果你是一位数据结构专家, 你就会知道, 实现 `sets` 的方法太多了, 其中一种便是在底层采用 `linked lists`。而你又刚好知道, 标准程序库有一个 `list` `template`, 于是你决定复用它。

更明确地说, 你决定让你那个萌芽中的 `Set` `template` 继承 `std::list`。也就是让 `Set<T>` 继承 `list<T>`。毕竟在你的实现理念中 `Set` 对象其实是个 `list` 对象。你于是声明 `Set` `template` 如下:

```
template<typename T>                //将 list 应用于 Set。错误做法。
class Set: public std::list<T> { ... };
```

每件事看起来都很好, 但实际上有些东西完全错误。一如条款 32 所说, 如果 D 是一种 B, 对 B 为真的每一件事情对 D 也都应该为真。但 `list` 可以内含重复元素, 如果数值 3051 被安插到 `list<int>` 两次, 那个 `list` 将内含两笔 3051。Set 不可以内含重复元素, 如果数值 3051 被安插到 `Set<int>` 两次, 这个 `set` 只内含一笔 3051。因此“Set 是一种 `list`”并不为真, 因为对 `list` 对象为真的某些事情对 `Set` 对象并不为真。

由于这两个 `classes` 之间并非 **is-a** 的关系, 所以 `public` 继承不适合用来塑模它们。正确的做法是, 你应当了解, `Set` 对象可根据一个 `list` 对象实现出来:

```
template<class T>                    //将 list 应用于 Set。正确做法。
class Set {
public:
    bool member(const T& item) const;
    void insert(const T& item);
    void remove(const T& item);
    std::size_t size() const;
private:
    std::list<T> rep;                //用来表述 Set 的数据
};
```

Set 成员函数可大量倚赖 list 及标准程序库其他部分提供的机能来完成，所以其实现很直观也很简单，只要你熟悉以 STL 编写程序：

```
template<typename T>
bool Set<T>::member(const T& item) const
{
    return std::find(rep.begin(), rep.end(), item) != rep.end();
}

template<typename T>
void Set<T>::insert(const T& item)
{
    if (!member(item)) rep.push_back(item);
}

template<typename T>
void Set<T>::remove(const T& item)
{
    typename std::list<T>::iterator it = //见条款 42 对
        std::find(rep.begin(), rep.end(), item); // "typename" 的讨论
    if (it != rep.end()) rep.erase(it);
}

template<typename T>
std::size_t Set<T>::size() const
{
    return rep.size();
}
```

这些函数如此简单，因此都适合成为 inlining 候选人。但请记住，在做出任何与 inlining 有关的决定之前，应该先看看条款 30。

也许有人主张，如果 Set 接口遵循 STL 容器的协议，就更符合条款 18 对设计接口的警告：“让它容易被正确使用，不易被误用”。但是这儿如果要遵循那些协议，需得为 Set 添加许多东西，那将模糊了它和 list 之间的关系。由于 Set 和 list 之间的关系是本条款的重点，所以我们以教学清澈度交换 STL 兼容性。此外，Set 接口也不该造成“对 Set 而言无可置辩的权利”黯然失色，那个权利是指它和 list 间的关系。这关系并非 **is-a**（虽然最初似乎是），而是 **is-implemented-in-terms-of**。

请记住

- 复合（composition）的意义和 public 继承完全不同。
- 在应用域（application domain），复合意味 **has-a**（有一个）。在实现域（implementation domain），复合意味 **is-implemented-in-terms-of**（根据某物实现出）。

条款 39: 明智而审慎地使用 private 继承

Use private inheritance judiciously.

条款 32 曾经论证过 C++ 如何将 public 继承视为 **is-a** 关系。在那个例子中我们有个继承体系, 其中 class Student 以 public 形式继承 class Person, 于是编译器在必要时刻 (为了让函数调用成功) 将 Students 暗自转换为 Persons。现在我再重复该例的一部分, 并以 private 继承替换 public 继承:

```
class Person { ... };
class Student: private Person { ... };    //这次改用 private 继承
void eat(const Person& p);                //任何人都会吃
void study(const Student& s);              //只有学生才在校学习

Person p;                                 //p 是人
Student s;                                //s 是学生

eat(p);                                   //没问题, p 是人, 会吃。
eat(s);                                   //错误! 吓, 难道学生不是人?!
```

显然 private 继承并不意味 **is-a** 关系。那么它意味什么?

“哇喔!”你说,“在我们探讨其意义之前, 可否先搞清楚其行为。到底 private 继承的行为如何呢?” 唔, 统御 private 继承的首要规则你刚才已经见过了: 如果 classes 之间的继承关系是 private, 编译器不会自动将一个 derived class 对象 (例如 Student) 转换为一个 base class 对象 (例如 Person)。这和 public 继承的情况不同。这也就是为什么通过 s 调用 eat 会失败的原因。第二条规则是, 由 private base class 继承而来的所有成员, 在 derived class 中都会变成 private 属性, 纵使它们在 base class 中原本是 protected 或 public 属性。

够了, 现在让我们开始讨论其意义。Private 继承意味 **implemented-in-terms-of** (根据某物实现出)。如果你让 class D 以 private 形式继承 class B, 你的用意是为了采用 class B 内已经备妥的某些特性, 不是因为 B 对象和 D 对象存在有任何观念上的关系。private 继承纯粹只是一种实现技术 (这就是为什么继承自一个 private base class 的每样东西在你的 class 内都是 private: 因为它们都只是实现枝节而已)。借用条款 34 提出的术语, private 继承意味只有实现部分被继承, 接口部分应略去。如果 D 以 private 形式继承 B, 意思是 D 对象根据 B 对象实现而得, 再没有其他意涵了。Private 继承在软件“设计”层面上没有意义, 其意义只及于软件实现层面。